EECS 221: Machine Learning for Science

# Data–Compute Trade-offs in Neural PDE Forecasting

Suha Kagan Kose

Mid-Project Report

**Project Goal.** Neural PDE forecasting models must be useful under limited data and compute. This project compares how accurately and efficiently three architecture families (CNextU-Net, TFNO and AViT) learn Rayleigh–Bénard convection when training data and GPU time are limited. This check-in covers the most constrained setting of the study: 10% of the training data (140 trajectories, four per regime) and 1 or 4 hours of training on one A100 GPU. The 100% runs follow in the next stage of the project (Section 7).

**Milestones.** The abstract set three milestones for this check-in; all of these were carried out as planned.

1. **Data pipeline validated against The Well's released checkpoints.** Our data loading and metric code reproduce the paper's published test VRMSE for the released CNextU-Net and TFNO (0.6688 against 0.6699 and 0.6634 against 0.6566, within 0.2% and 1.0%), and recover rollout errors in the same reported range ("$> 10$"). The released TFNO needs the forward pass it was trained with, which differs from the corrected one our runs use (Section 1, Appendix D).
2. **Budgets fixed from pilot timings.** Batch 64, fp32, peak learning rate $10^{-3}$ for every model (from an equal-budget search), and 4-hour budgets measured in active GPU training time, with allocated GPU time reported separately.
3. **First 10%-data results.** All six 10% runs (3 models $\times$ 2 seeds) are trained and evaluated on the full test set at 1 and 4 hours. Reported together with what each model costs to train and to use (Section 4). A revised plan is in Section 7.

## 1 Setup

**Physical system** (Fig. 1). Rayleigh–Bénard convection is a horizontal layer of fluid heated from below and cooled from above. Warm fluid near the bottom is lighter than the cold fluid above it; once this buoyancy forcing is strong enough to overcome the damping by viscosity and heat diffusion, the layer overturns into rising warm and sinking cold plumes, with thin thermal boundary layers along both walls. It is the classical model of buoyancy-driven convection, as in the atmosphere, the oceans, stellar interiors and the Earth's mantle, and a standard test problem for turbulence and heat transfer. We consider its two-dimensional form: buoyancy $b$ (the temperature variable), velocity $(u, w)$ and pressure $p$ in a $4\times1$ box, periodic in $x$, between a hot bottom wall ($b = 1$) and a cold top wall ($b = 0$). Two dimensionless numbers set a regime: the *Rayleigh number* Ra, the strength of the buoyancy forcing relative to viscous and thermal damping, and the *Prandtl number* Pr, how fast momentum diffuses relative to heat (definitions in Appendix A). A larger Rayleigh number means stronger, more turbulent convection with finer plumes and thinner boundary layers. Started from a fluid at rest, convection first has to grow out of the nearly still state (*onset*) and then settles into statistically steady, *developed* convection.

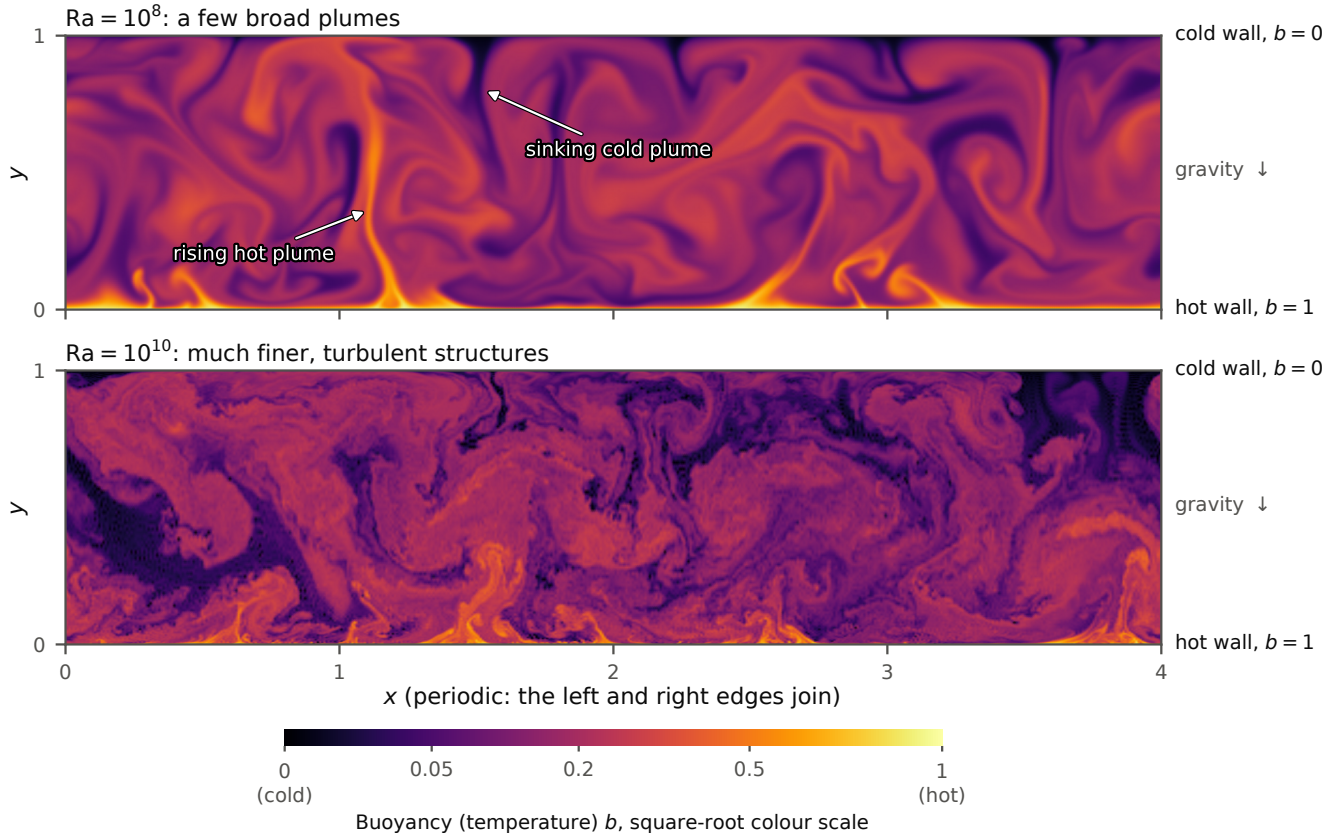



**Figure 1:** The physical system: true buoyancy (temperature) in two trajectories of The Well's test split, both at $\mathrm{Pr} = 1$, at frame 83 of 200 (developed convection). The bottom wall is hot ($b = 1$), the top wall is cold ($b = 0$), gravity points down and $x$ is periodic. Hot plumes rise from the bottom wall and cold plumes sink from the top; at the higher Rayleigh number the plumes break up into much finer structures.

**Data.** `rayleigh_benard` from The Well [1, 13]: 1,750 trajectories, 50 in each of 35 regimes (five Rayleigh numbers from $10^6$ to $10^{10}$ × seven Prandtl numbers from 0.1 to 10; Appendix A); 367 GB in all. Each trajectory starts from rest, with a linear buoyancy profile plus a small random perturbation; within a regime, trajectories differ in the profile's amplitude and the perturbation's random seed. Each has 200 frames, 0.25 simulation time units apart, of the four fields on a $512 \times 128$ grid: uniform and periodic in $x$, and on Chebyshev–Gauss nodes in $y$, which cluster near the walls to resolve the boundary layers (the files store a uniform placeholder for $y$; we identified the nodes from the data, Appendix C). We keep The Well's official split, which assigns whole trajectories: 1,400 for training, 175 for validation and 175 for testing (40 / 5 / 5 in each regime). The study trains on nested subsets of the training split; this report uses the 10% subset (140 trajectories, four of the 40 in each regime, 29 GB) and evaluates on the full test split. Fields are normalised with The Well's training statistics.

**Task.** From four consecutive frames of buoyancy, pressure and both velocity components, a model predicts the next frame; longer forecasts come from feeding its predictions back in (autoregressive rollout). Each trajectory gives 196 one-step windows.

**Models.** The Well's own implementations, trained from random initialisation, with matched parameter counts (Table 1). All three use GELU activations. For TFNO, released-checkpoint reproduction and training with the current code must be distinguished: the paper-era code skipped TFNO's Fourier layers, an error fixed upstream in November 2025. We scored the released TFNO weights with the forward pass they were trained with, and our own TFNO, trained with the fixed code, uses its Fourier layers (Appendix D).

| Model | Family | Parameters | Relation to released checkpoint |
|---|---|---|---|
| CNextU-Net [2] | ConvNeXt U-Net, 4 stages | 18.58M | same architecture |
| TFNO [5, 3] | Fourier operator, 16×16 modes | 19.28M | same configuration, fixed forward |
| AViT [4] | axial-attention ViT | 18.57M | none released; width 384 → 432 |

**Table 1:** The three model families.

**Training.** Every model is trained the same way: AdamW on the one-step MSE of the normalised fields, batch 64, fp32, on one A100. The learning rate warms up over 500 steps to a peak of $10^{-3}$, then decays to zero along a cosine over four hours of active GPU training. Each run is evaluated at one and four hours, the two budgets of the study's compute axis. The paper trained for 12 hours on an H100 with its own learning rates and schedule, so our runs are a different experiment, not a replication (Appendix B).
**Seeds.** Each configuration is trained with two seeds (initial weights and data order) on the same 10% subset.

**Method and baselines.** The method is a controlled comparison: the three families are trained from scratch with one recipe on the same data and budgets, and compared on accuracy and on cost to train and to use. Two references are shown next to them in the results:

- *Persistence*, the baseline to beat, predicts that the next frame equals the last input frame. It needs no training and almost no computation, so a model is worth its training cost only where it beats persistence. It is hard to beat where the flow changes slowly, as during onset.
- The Well's released *CNextU-Net*, trained on all the training data for 12 hours on an H100, shows what this architecture reaches with ten times the data and a longer schedule. It is a reference point, not a baseline to beat. It also validated our pipeline (Section 2).

The released TFNO is not used as a reference. It was trained with a version of The Well's code that skipped its Fourier layers, so it is effectively a per-pixel network that does little more than copy the last frame (Appendix D). A numerical-solver baseline and three cheaper baselines are planned for the final report (Section 6.1).

**VRMSE.** For one predicted frame and one field $f$, The Well's metric code computes

$$\mathrm{VRMSE}_f = \sqrt{\frac{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_{f,i} - y_{f,i})^2}{s_f^2 + \varepsilon}}, \qquad s_f^2 = \frac{1}{N-1}\sum_{i=1}^{N}(y_{f,i} - \bar{y}_f)^2, \qquad \varepsilon = 10^{-7},$$

over the $N = 512 \times 128$ grid points. It compares the size of the error with how much the true frame varies across the grid; $\varepsilon$ only keeps it from dividing by zero. Scores are averaged over the four fields (buoyancy $b$, pressure $p$, velocities $u$, $w$) and over windows (one-step: all 196 windows of each of the 175 test trajectories; rollouts: at each step). The paper first rescales each field by its training mean and standard deviation (*paper protocol*); we score the fields as stored in the data (*physical-field VRMSE*). During onset, when the fluid is nearly still and the denominator is close to zero: $\varepsilon$ then holds the physical-field score down but barely affects the rescaled fields, so the paper protocol gives much larger scores. To avaoid this, we lead with the physical-field VRMSE, which these first frames dominate less, and report the paper protocol wherever we compare with the paper (details in Appendix C).

**Failure horizon.** For a rollout, the first step whose field-mean physical-field VRMSE exceeds a threshold (1, 2 or 5, all reported). A field held constant at the target's spatial mean scores VRMSE $\approx 1$, so a forecast above threshold 1 says less about the spatial pattern than that mean alone. Rollouts start at frames 20, 80 and 140 of every test trajectory and run 30 steps. We report the fraction of rollouts that stay below each threshold through step 30 (Table 4) and, for threshold 1, its value at every step (Kaplan–Meier

survival; Fig. 5b). Additionally, the kinetic-energy spectrum and the inference-time measurements, that are included in this report are also defined in Appendix C.

## 2 Milestone 1: Pipeline Validated Against the Released Checkpoints

We loaded The Well's released CNextU-Net and TFNO weights and scored them with our own data path and metric on every window of the 175 test trajectories (Table 2). The released TFNO also reproduces its published value, but only with the forward pass it was trained with. A bug in The Well's code at the time skipped TFNO's Fourier layers, so the released model is effectively a per-pixel network: each output pixel depends only on the same pixel's input frames, and it cannot move structures across the grid. It therefore learned little more than copying the last frame, and its error tracks persistence (Fig. 2): slightly lower in developed convection, much higher during onset. Its full results are in Appendix D.

| | Paper-protocol one-step VRMSE | | Rollout from frame 0, steps 6–12 / 13–30 | |
|---|---|---|---|---|
| Model | Ours (released weights) | Published | Ours | Published |
| CNextU-Net | 0.6688 | 0.6699 | 36 / 35 | $> 10$ / $> 10$ |
| TFNO* | 0.6634 | 0.6566 | 142 / 169 | $> 10$ / $> 10$ |

**Table 2:** The released CNextU-Net and TFNO scored by our pipeline against the paper's Tables 2 and 3.

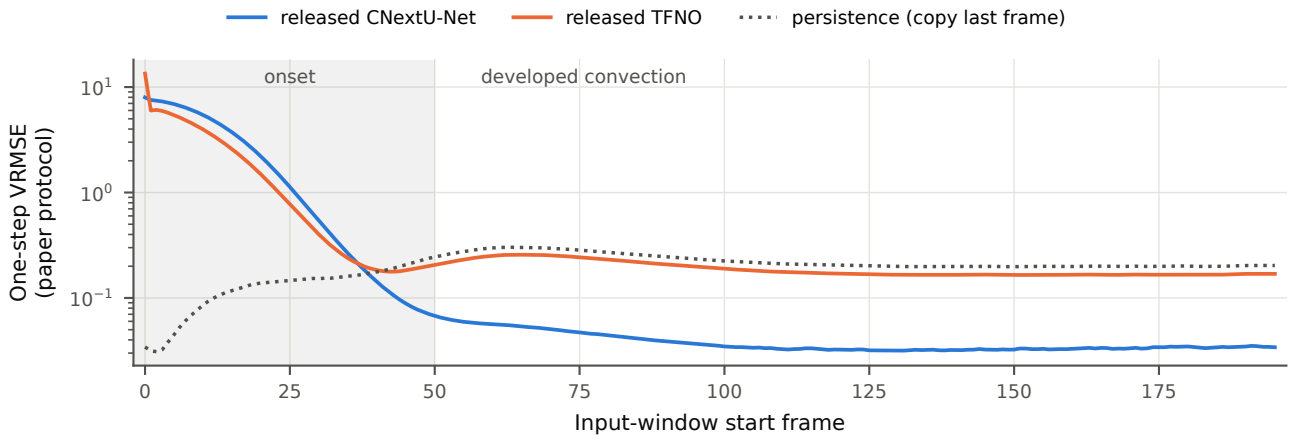



**Figure 2:** Released checkpoints' one-step VRMSE by window on the test set under the paper protocol, mean over the 175 trajectories. Shaded: windows whose first input frame is below 50, reported as *onset*; the others are *developed*. The cut at frame 50 is operational, chosen where the released checkpoints' error levels off on the validation split; it is not a transition verified for every regime. Persistence copies the last input frame. The released TFNO tracks persistence because of the paper-era bug (Section 2, Appendix D).

## 3 Milestone 2: Budgets Determined from Pilot Timings

**Learning rate.** The paper's rates ($5 \cdot 10^{-4}$ for CNextU-Net, $10^{-4}$ for TFNO) were chosen for 12-hour runs. With one to four hours a model takes far fewer optimizer steps, and a small learning rate that pays off over a long run may not get far enough in a short one, so we tested whether a higher rate suits our budgets. An equal-budget search (20 minutes of GPU training per run, 10% data, validation only) chose $10^{-3}$ for all three models (Appendix Table 11). At this budget the higher rate was clearly better than the paper's: 0.408 against 0.469 for CNextU-Net at $5 \cdot 10^{-4}$, and 0.302 against 0.603 for TFNO at $10^{-4}$.

**Batch size and precision.** Batch 64 (The Well's) fits every model in fp32 on an A100 80 GB; CNextU-Net peaks at 64.8 GB of memory reserved by PyTorch (60.9 GB allocated, Table 6). The current TFNO implementation requires fp32 FFT operations, so all models use fp32 for this comparison.

**Training budget.** A run's 1-hour or 4-hour budget counts only *active GPU training time*: the clock runs while the GPU computes training steps (forward pass, backward pass and weight update) and stops otherwise (Appendix E gives the exact timer boundaries). We count GPU time rather than elapsed time because data loading cannot keep up with every model: the shared storage delivers at most ∼98 training samples per second into one node, while TFNO and AViT consume 116 and 330 per second on the GPU (Fig. 10). An elapsed-time budget would therefore partly measure the storage rather than the model. The time a run spends waiting for data, starting up, validating and saving checkpoints is recorded separately; together with the training time it makes up the GPU time the run holds (*allocated GPU time*; Appendix Table 15).

## 4 Milestone 3: First Results Under Limited Data and Compute

This section answers the project's question for its most constrained setting: how accurate each family becomes with 10% of the training data and 1 or 4 hours of training, and what it costs to train and to use.

**Training** (Fig. 3). The six runs completed their 4-hour budgets (Appendix Table 14). On a fixed validation sample of 64 windows (Appendix E), AViT and CNextU-Net reach similar error after 4 hours; TFNO's loss plateaus after ∼1.5 hours. CNextU-Net's validation error peaks near one hour while its learning rate is still high; this temporal association does not establish the learning rate as the cause.

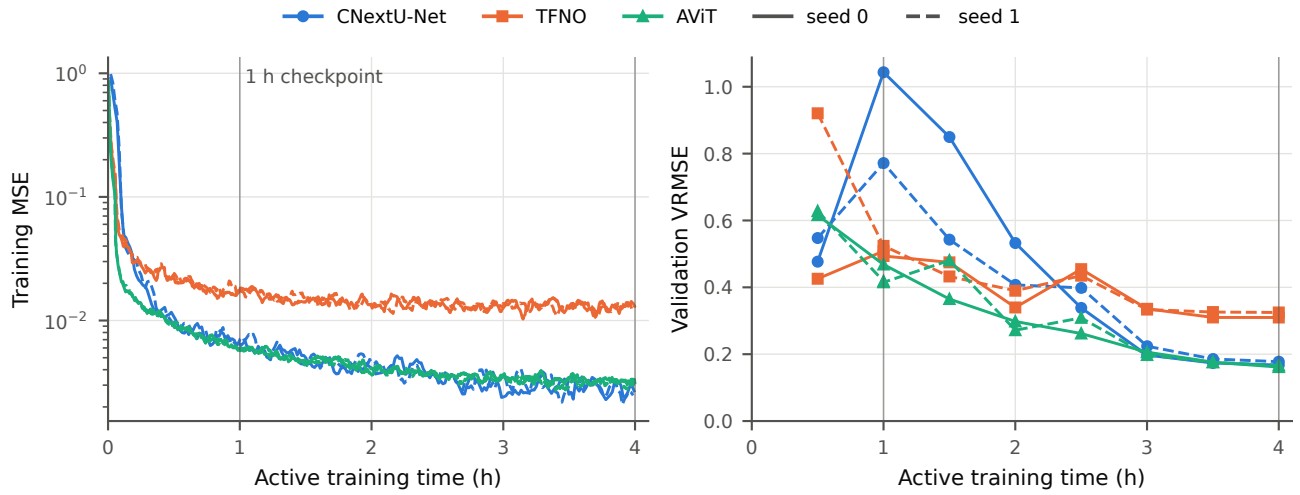



**Figure 3:** Training MSE in normalised units (left; smoothed over 3 minutes of training) and validation VRMSE on the 64 fixed windows, physical fields (right), against active training time, 10% data. Solid: seed 0; dashed: seed 1. Vertical lines: the 1-hour and 4-hour checkpoints.

**Test evaluation** (all 175 test trajectories, both seeds; Table 3, Fig. 4). After 4 hours AViT and CNextU-Net have similar one-step error over all windows (0.134 and 0.136) and TFNO a higher one (0.209), but the two phases separate them: in developed convection CNextU-Net is clearly the most accurate (0.050, against 0.075 for AViT and 0.109 for TFNO), consistent with U-Net variants being among the strongest baselines in neural PDE benchmarks [6, 1], while AViT is the most accurate during onset. The across-seed standard deviation of the all-window VRMSE is at most 0.006 (seeds only; the subset is fixed). The released CNextU-Net, trained on all the data for longer, remains better in developed convection (0.038

against 0.050). This gap is not a measure of what ten times more data buys, because the two also differ in hardware, learning rate and schedule (Appendix B). The 100% runs will measure the effect of data with everything else fixed. Persistence is better than every model during onset on the four-field mean, a velocity effect (Table 5), and 2–4.5 times worse in developed convection.

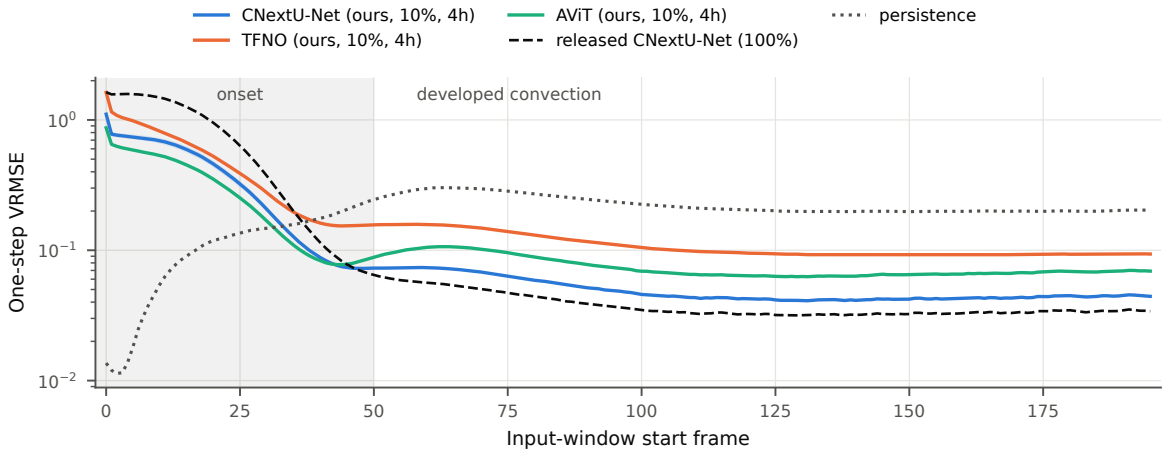



**Figure 4:** One-step test VRMSE (physical fields) by window, mean over the 175 test trajectories: our 10% models at 4 hours (seed mean; band: range over seeds), the released CNextU-Net trained on all the data, and persistence. Shaded: onset windows (first input frame $< 50$; see Fig. 2).

| | Physical-field one-step VRMSE | | | Paper protocol |
|---|---|---|---|---|
| Model (10% data, checkpoint) | all windows | onset | developed | (all windows) |
| CNextU-Net, 4 h | 0.136 ± 0.006 | 0.387 ± 0.025 | **0.050** ± 0.000 | 0.349 ± 0.022 |
| TFNO, 4 h | 0.209 ± 0.001 | 0.502 ± 0.003 | 0.109 ± 0.000 | 0.488 ± 0.001 |
| AViT, 4 h | **0.134** ± 0.001 | **0.308** ± 0.003 | 0.075 ± 0.000 | **0.298** ± 0.002 |
| CNextU-Net, 1 h | 0.728 ± 0.176 | 2.618 ± 0.681 | **0.081** ± 0.003 | 2.345 ± 0.600 |
| TFNO, 1 h | 0.473 ± 0.078 | 1.505 ± 0.305 | 0.120 ± 0.000 | 1.362 ± 0.249 |
| AViT, 1 h | **0.394** ± 0.036 | **1.299** ± 0.140 | 0.085 ± 0.000 | **1.171** ± 0.125 |
| Released CNextU-Net (100%, paper) | 0.224 | 0.766 | 0.038 | 0.669 |
| Persistence | 0.199 | 0.121 | 0.226 | 0.202 |

**Table 3:** One-step test error: mean ± standard deviation over the two seeds (0.000: below 0.0005). Onset: first input frame $< 50$; developed: $\geq 50$. Bold: best of our models at each budget.

**Rollouts** (Table 4, Fig. 5). In a rollout, each prediction becomes the model's next input. Our models were trained only to predict one step ahead from true frames, so in a rollout they receive their own slightly wrong outputs, inputs they never saw in training [9], and their errors can grow from step to step [10]. The best one-step model is therefore not necessarily the best over a rollout.

CNextU-Net is the most stable of our models. Started from developed flow (frames 80 and 140), its error after 30 steps is 0.53 and 0.40, below persistence's 0.86 and 0.54. Over all three start frames, 89% of its rollouts stay below VRMSE 1 for all 30 steps. AViT has about the same one-step error as CNextU-Net but degrades fastest. After 30 steps from developed flow its error is 0.80 and 0.65, on average no better than persistence (0.72 against 0.70); only 63% of its rollouts stay below VRMSE 1; and from frame 20, during onset, its error reaches 5.4. TFNO lies in between.

Persistence keeps the most rollouts below the thresholds, but not because it forecasts well: its error quickly levels off near 0.7 instead of growing. Over the first steps from developed flow it is 2–5 times less accurate than every model (Fig. 5a). Under the paper's protocol (rollouts from frame 0, normalised

fields), all three models score above 10, as every model does in the paper; our CNextU-Net (15 and 30 over steps 6–12 and 13–30) is below the released one (36 and 35).

| Model | VRMSE after 30 steps from frame 20 / 80 / 140 | Fraction staying below threshold through step 30: 1 / 2 / 5 | Paper protocol, steps 6–12 / 13–30 (normalised) |
|---|---|---|---|
| CNextU-Net, 4 h | 1.78 / **0.53** / **0.40** | **0.89** / **0.94** / **0.98** | **15** / **30** |
| TFNO, 4 h | 1.79 / 0.59 / 0.45 | 0.75 / 0.85 / 0.91 | 51 / 77 |
| AViT, 4 h | 5.42 / 0.80 / 0.65 | 0.63 / 0.80 / 0.87 | 75 / 167 |
| CNextU-Net, 1 h | 3.60 / 1.09 / 1.04 | 0.34 / 0.74 / 0.79 | 202 / 148 |
| TFNO, 1 h | **2.67** / **0.90** / **0.78** | **0.60** / **0.77** / **0.86** | **81** / **116** |
| AViT, 1 h | 4.67 / 1.11 / 0.98 | 0.34 / 0.74 / 0.83 | 129 / 184 |
| Released CNextU-Net (100%) | | | 36 / 35 |
| Persistence | 0.68 / 0.86 / 0.54 | 0.94 / 1.00 / 1.00 | 0.27 / 0.50 |

**Table 4:** Rollouts on the test set, physical-field VRMSE unless stated, mean over the two seeds. Middle column: fraction of the 525 rollouts (175 trajectories × 3 start frames) whose error stays below each threshold at every step through step 30. Right: the paper's Table 3 protocol (published: $> 10$ in both windows); bold: best of our models at each budget.

**With only one hour of training** (Tables 3 and 4; the four-hour runs' checkpoints after one hour). The ranking differs from four hours, and no model is best on every count. AViT has the lowest one-step error over all windows (0.39, against 0.47 for TFNO and 0.73 for CNextU-Net) and during onset; in developed convection CNextU-Net (0.081) and AViT (0.085) are close and TFNO trails (0.120). Over rollouts TFNO is the most stable: 60% of its rollouts stay below VRMSE 1 through step 30, against 34% for the other two, and from developed flow it stays more accurate than persistence for the first 21 steps, against 14 for CNextU-Net and 12 for AViT. CNextU-Net, the best model after four hours, is the least reliable after one: its two seeds differ most (all-window VRMSE 0.85 and 0.60), and none of one seed's rollouts stays below VRMSE 1, against 69% for the other. With one hour of training, AViT is the better choice for one-step predictions and TFNO for multi-step forecasts; after 30 steps none of the three is more accurate than persistence.

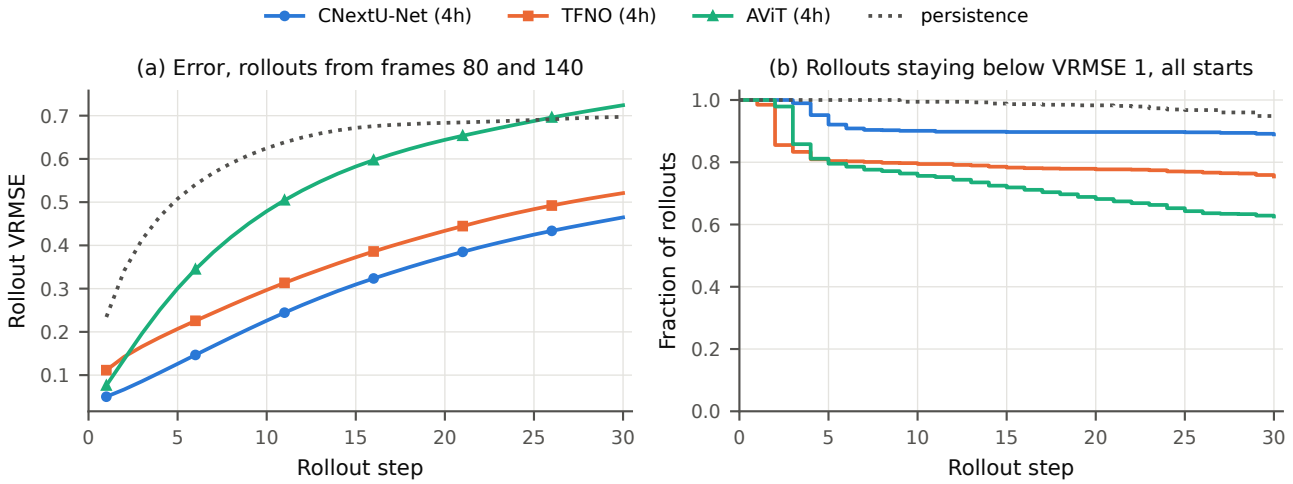



**Figure 5:** Rollouts of our 10% models at 4 hours (seed mean) and persistence. (a) Physical-field VRMSE at each step, mean over the rollouts from frames 80 and 140 (developed convection). (b) Fraction of all 525 rollouts that have stayed below VRMSE 1 up to each step (Kaplan–Meier survival).

**Field-wise error** (Table 5). The four-field mean hides where the error lies. In developed convection buoyancy is the hardest field for every model (CNextU-Net: 0.092, against 0.024 for pressure and about 0.04 for each velocity component), consistent with the sharp gradients of its thin plumes and thermal

boundary layers. During onset the picture reverses: buoyancy and pressure errors are small, and the mean comes almost entirely from velocity (CNextU-Net: 0.86 and 0.64), whose near-zero spatial variance magnifies relative error (Section 2). On buoyancy all three models beat persistence even during onset (0.031–0.045 against 0.066), and CNextU-Net and AViT also on pressure, so the models' onset deficit against persistence is a velocity effect. After 30 steps from developed flow, CNextU-Net and TFNO stay below persistence on every field, while AViT is worse than persistence on horizontal velocity (1.06 against 0.93), level on vertical velocity and pressure, and better only on buoyancy. After one hour, CNextU-Net's large all-window error is almost entirely onset velocity (6.8 for $u$); in developed convection it is best or tied on three of the four fields.

| | One step, developed | | | | One step, onset | | | | 30 steps from frames 80, 140 | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Model | $b$ | $p$ | $u$ | $w$ | $b$ | $p$ | $u$ | $w$ | $b$ | $p$ | $u$ | $w$ |
| CNextU-Net, 4 h | **0.092** | **0.024** | **0.039** | **0.044** | 0.032 | 0.012 | 0.863 | 0.639 | **0.384** | **0.293** | **0.598** | **0.584** |
| TFNO, 4 h | 0.170 | 0.054 | 0.096 | 0.116 | 0.045 | 0.016 | 1.164 | 0.783 | 0.466 | 0.307 | 0.665 | 0.646 |
| AViT, 4 h | 0.119 | 0.039 | 0.061 | 0.080 | **0.031** | **0.008** | **0.651** | **0.542** | 0.497 | 0.419 | 1.059 | 0.922 |
| CNextU-Net, 1 h | **0.126** | **0.046** | 0.079 | **0.072** | 0.051 | 0.034 | 6.829 | 3.557 | 1.017 | 0.729 | 1.280 | 1.249 |
| TFNO, 1 h | 0.180 | 0.062 | 0.108 | 0.128 | 0.057 | 0.029 | 3.745 | 2.190 | 0.925 | **0.603** | **0.930** | **0.908** |
| AViT, 1 h | 0.129 | **0.046** | **0.074** | 0.088 | **0.040** | **0.018** | **3.092** | **2.047** | **0.837** | 0.652 | 1.484 | 1.209 |
| Released CNextU-Net | 0.069 | 0.020 | 0.030 | 0.034 | 0.024 | 0.011 | 1.465 | 1.563 | – | – | – | – |
| Persistence | 0.272 | 0.122 | 0.245 | 0.264 | 0.066 | 0.014 | 0.202 | 0.201 | 0.531 | 0.408 | 0.934 | 0.916 |

**Table 5:** Field-wise test VRMSE (physical fields, mean over the two seeds): buoyancy $b$, pressure $p$, horizontal and vertical velocity $u$, $w$. Our models: 10% data; the released CNextU-Net: 100%. Bold: best of our models at each budget.

**Accuracy against cost** (Fig. 6, Table 6). Because data and compute are the limiting resources, we set each model's accuracy against what it costs to train and to use.

- *Training time* (Fig. 6a). From one to four hours the developed-convection error falls by 38% for CNextU-Net (0.081 to 0.050), 12% for AViT (0.085 to 0.075) and 9% for TFNO (0.120 to 0.109). At one hour CNextU-Net and AViT are close; CNextU-Net pulls ahead only with more training, while TFNO has nearly plateaued.

- *Cost to train* (Fig. 6b). Every run trained for the same 4 hours, but the GPU time it held differed. AViT is the most efficient on the GPU (six times as many optimizer steps as CNextU-Net, a seventh of the memory), but it consumes 330 samples per second, more than our shared storage delivers (∼98; Fig. 10). Each AViT run therefore held a GPU for 11–12 hours, against 6.9–7.1 for TFNO and 5.2–6.0 for CNextU-Net. On faster storage this ranking would change; the demand per GPU-second is the portable number. CNextU-Net's 61 GB of training memory at batch 64 needs an 80 GB GPU or a smaller batch.

- *Cost to use* (Fig. 6c). For a scientist running forecasts, inference is the recurring cost. Every model predicts the next $512 \times 128$ frame in 4–8 ms and a 30-step forecast (7.5 time units) in 135–249 ms, using under 0.5 GB of GPU memory. For one forecast at a time TFNO is the fastest; AViT takes 1.2 times and CNextU-Net 1.8 times as long. CNextU-Net's extra 114 ms per forecast buys a 30-step error of 0.47 instead of TFNO's 0.52, while AViT is slower than TFNO and, after 30 steps, no more accurate than persistence. The ranking changes for many forecasts at once: in batches of 64 the forward pass costs 0.89 ms per sample for AViT, 3.15 ms for TFNO and 6.74 ms for CNextU-Net, so for ensembles or many initial conditions AViT is 3.5 times cheaper per forecast than TFNO and 7.6 times cheaper than CNextU-Net. For context, The Well reports that generating the data with the Dedalus solver took about 6,000 to 50,000 s per parameter setting on CPU nodes [13]. That figure

covers many trajectories with an unstated parallel layout, so it is not comparable to a per-forecast time; the solver baseline in the plan would measure that comparison directly.

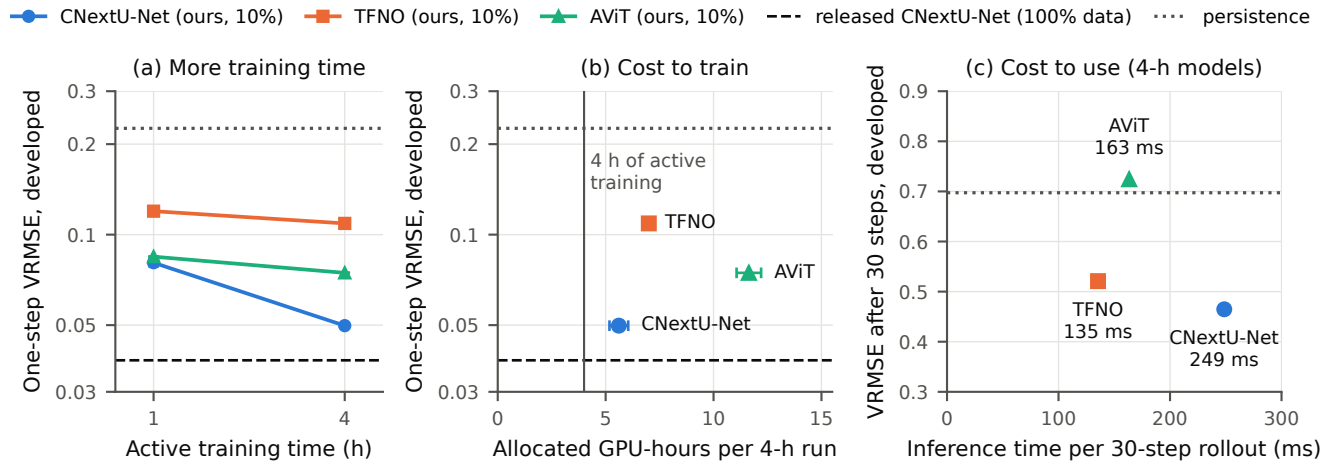



**Figure 6:** Accuracy against cost, 10% data (seed mean; bars: range over seeds). (a) One-step test VRMSE in developed convection at the 1-hour and 4-hour checkpoints. (b) The same error after 4 hours against the GPU time each run held, including data waits and overhead; vertical line: the 4 hours of active training. (c) Rollout VRMSE after 30 steps from frames 80 and 140 against the time to compute one 30-step forecast (batch 1, A100). Dashed: the released CNextU-Net (100% data); dotted: persistence.

| | Training (4-hour budget, batch 64) | | | | Inference | | | |
|---|---|---|---|---|---|---|---|---|
| Model | Optimizer steps | Epochs | Peak memory, alloc. (reserved) | GPU time held per run | per step, batch 1 | 30 steps, batch 1 | per sample, batch 64 | Memory, batch 1 |
| CNextU-Net | 12,122–12,194 | 28 | 60.9 (64.8) GB | 5.2–6.0 h | 8.3 ms | 249 ms | 6.74 ms | 0.31 GB |
| TFNO | 26,339–26,352 | 61 | 48.6 (56.9) GB | 6.9–7.1 h | 4.4 ms | 135 ms | 3.15 ms | 0.44 GB |
| AViT | 74,978–75,060 | 175 | 8.8 (8.9) GB | 11.1–12.2 h | 5.4 ms | 163 ms | 0.89 ms | 0.23 GB |

**Table 6:** Cost to train and to use, on one A100 80 GB in fp32. Peak training memory: allocated by tensors (reserved by PyTorch's allocator). GPU time held: allocated time of the training job, including data waits, start-up, validation and checkpoints (Appendix E). Per sample, batch 64: forward pass in the training profile (with gradient tracking, an upper bound for inference). Inference memory: peak allocated at batch 1.

**Physical fidelity** (4-hour checkpoints; Fig. 7). The kinetic-energy spectrum shows at which scales each model's errors lie. It is a complementary diagnostic, not the paper's binned spectral error, and it describes how energy is distributed over scales, not where structures are: a model can match the spectrum with plumes in the wrong places. The values below are medians over the 35 regimes and both seeds of the predicted-to-true ratio, taken per wavenumber at steps 1, 10 and 30 (the steps evaluated). CNextU-Net stays within 0.80–1.57 below $k_x = 100$ and reaches 190–530 at the highest wavenumbers. TFNO is too smooth: between $k_x \approx 25$ and 100 its ratio falls to 0.33 after one step and 0.09 after 30, and it reaches 9–50 at the highest wavenumbers. AViT stays within 0.97–1.34 below $k_x = 100$ after one step, but after 30 steps it ranges from 0.55 to 21 there and reaches about $5 \times 10^4$ at the highest wavenumbers. The TFNO deficit is consistent with its spectral layers keeping only the 16 lowest Fourier modes in each direction ($k_x \lesssim 25$) [5], and AViT's growth with the patch-shaped artifacts in Fig. 8; neither is established as the cause. The excess small-scale energy of CNextU-Net and AViT is typical of surrogates trained with MSE, which weights each scale by its energy and so barely constrains low-energy small scales (the spectral bias of neural networks [7]); such surrogates are known to neglect non-dominant, often high-frequency, components [8]. Persistence repeats its last input field, so its predicted spectrum is fixed within a rollout;

changes in its ratio reflect the evolving true spectrum. Ratios at the smallest scales should also be read alongside absolute spectra, because very small true energy can magnify the ratio.

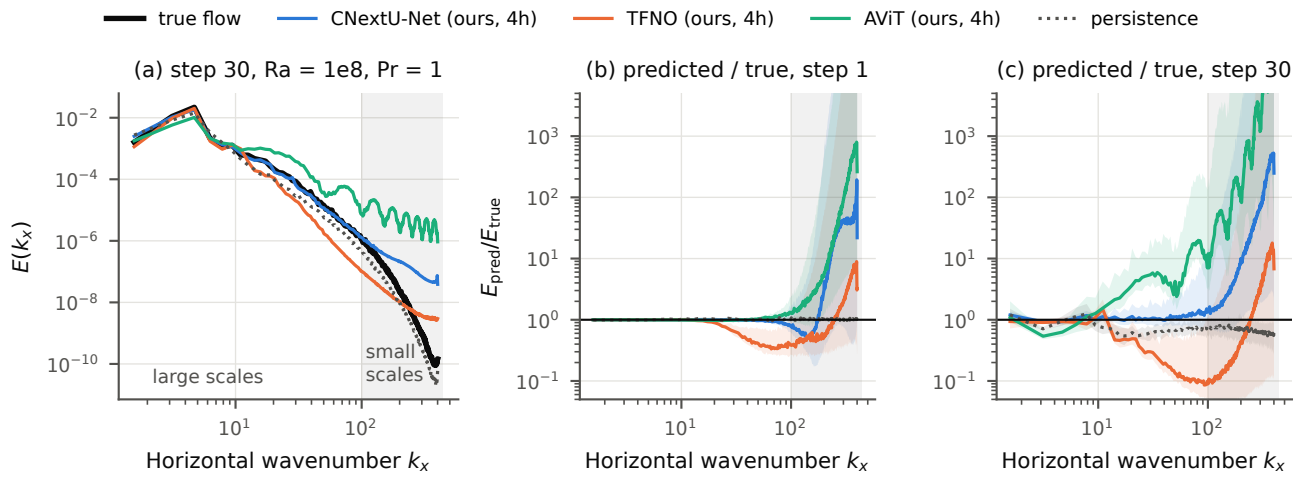



**Figure 7:** Horizontal kinetic-energy spectra. (a) True and predicted spectrum after 30 rollout steps for one regime ($\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$). (b, c) Predicted over true spectrum after 1 and 30 steps, per wavenumber: median over the 35 regimes (band: middle 50%); 1 is exact. Shaded: small scales ($k_x > 100$).

**What the predictions look like** (Fig. 8; $\mathrm{Ra} = 10^{10}$ in Appendix G). The figure shows buoyancy, while the spectra are computed from velocity, so the two are related qualitative evidence rather than measurements of the same field. At $\mathrm{Ra} = 10^8$ CNextU-Net follows the true plumes through 30 steps. TFNO blurs them and builds a spurious warm layer along the bottom wall. By step 12 AViT shows rectangular artifacts at the size of its $16 \times 16$-pixel patches. At $\mathrm{Ra} = 10^{10}$ the true flow is far finer than any of the three reproduces; CNextU-Net keeps the large-scale plume pattern but smooths the small structures.

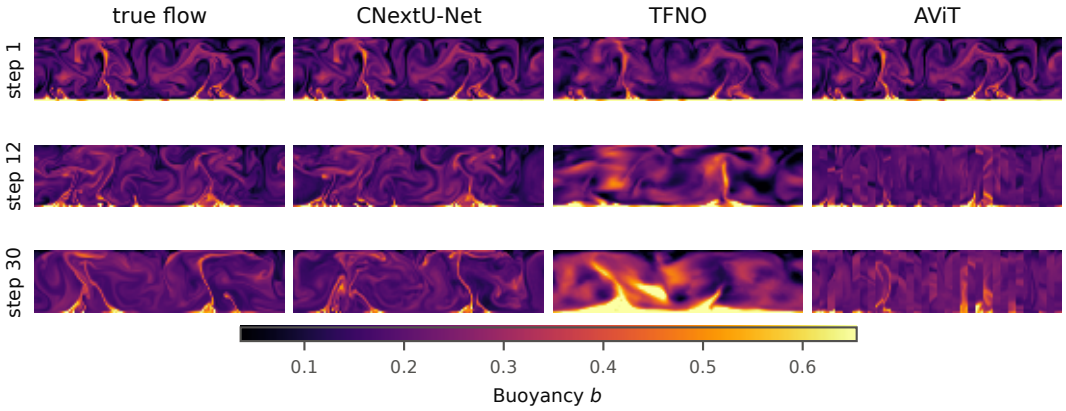



**Figure 8:** Buoyancy on a test trajectory at $\mathrm{Ra} = 10^8$, $\mathrm{Pr} = 1$, rolled out from frame 80: the true flow and our 10% / 4-hour models (seed 0) after 1, 12 and 30 steps. Bright: warm; dark: cold. One colour range is used for all panels: the 1st to 99th percentile of the true field away from the walls, so the hot bottom wall saturates.

**Choosing a model under limited data and compute** (Table 7). No family is best on every count. CNextU-Net is the most useful forecaster in developed convection at 10% data and 4 hours, at the price of the slowest inference and the most training memory; TFNO is the fastest for single forecasts but too smooth; AViT is the cheapest on the GPU, in training and for batched forecasts, but needs fast storage and does not stay accurate over a rollout.

| If the priority is | Best of the three | Trade-off |
|---|---|---|
| Accurate forecasts of developed convection that stay stable over 30 steps | CNextU-Net | Slowest inference (249 ms per 30 steps); 61 GB training memory at batch 64; gains most from longer training |
| Fastest single forecast (batch 1) | TFNO (135 ms per 30 steps) | Too smooth: loses energy between $k_x \approx 25$ and 100; largest one-step error in developed convection; gains least from longer training |
| Least training memory, most optimizer steps per GPU-hour, cheapest batched forecasts | AViT (8.8 GB; 0.89 ms per sample at batch 64) | Needs storage delivering ∼330 samples/s; rollouts degrade fastest, no better than persistence after 30 steps |
| Convection onset (first 50 frames) | None: persistence (four-field mean) | Every model is less accurate than copying the last frame on velocity, whose variance is near zero; on buoyancy all three are more accurate |
| Only one hour of training | AViT (one step); TFNO (rollouts) | AViT: lowest one-step error (0.39); TFNO: 60% of rollouts stay below VRMSE 1, against 34%; CNextU-Net varies most between seeds; none beats persistence after 30 steps |

**Table 7:** Which model to use under which constraint, from the 10% results, after four hours of training unless the row says otherwise. These hold for these configurations and budgets and may change with all the training data.

# 5 Findings Under Limited Data and Compute

The main findings at 10% data hold for these configurations and budgets (Table 7):

- **Accuracy.** After 4 hours CNextU-Net is the most accurate in developed convection (one-step VRMSE 0.050) and the most stable over 30-step rollouts. With a tenth of the data and 4 hours on an A100, its error is 1.3 times that of The Well's released CNextU-Net, trained on all the data for 12 hours on an H100 (0.038); the two also differ in schedule and hardware, so this is not an estimate of what more data buys. TFNO's predictions are too smooth; AViT matches CNextU-Net over all windows but degrades fastest over a rollout. In developed flow buoyancy is the hardest field for every model; the large onset errors come from velocity.
- **With only one hour of training** the ranking differs. AViT has the lowest one-step error and TFNO the most stable rollouts, while CNextU-Net, the best after four hours, varies most between seeds. CNextU-Net gains most from the extra three hours: its error in developed convection falls by 38%, AViT's by 12% and TFNO's by 9%.
- **Cost to train.** In the same 4 hours AViT takes six times as many optimizer steps as CNextU-Net with a seventh of its memory, but it reads data faster than our shared storage delivers, so a run held a GPU for 11–12 hours, against 5–6 hours for CNextU-Net.
- **Cost to use.** Each model forecasts 30 steps (7.5 simulation time units) in at most a quarter of a second on one A100, using under 0.5 GB of GPU memory. For one forecast at a time TFNO is the fastest (135 ms), then AViT (163 ms) and CNextU-Net (249 ms), so the most accurate model is 1.8 times slower than the fastest. For batched forecasts AViT is the cheapest per sample (0.89 ms per step, against 3.15 for TFNO and 6.74 for CNextU-Net).
- **Negative results.** No model beats copying the last frame (persistence) on velocity during convection onset (on buoyancy all three do), and after 30 steps from developed flow AViT is no better than persistence.
- **Rayleigh number and heat transport.** Error grows four- to tenfold from $\mathrm{Ra} = 10^6$ to $10^{10}$ for every model, and the released full-data model's advantage is confined to high Rayleigh numbers.

After one step CNextU-Net reproduces the Nusselt number best (2% median error); over a 30-step rollout only TFNO predicts the averaged heat transport better than persistence, and AViT predicts too little (Appendix F).

- **Caveats.** Two seeds on one fixed 10% subset do not measure how much the choice of trajectories matters. Training in fp32 for every model favours TFNO, since bf16 would speed up CNextU-Net (1.7 times) and AViT (2.2 times) but is not available for our TFNO. The learning rate is the best of a short search, at the top of the tested grid.

# 6 Issues and Decisions for the Final Submission

## 6.1 Decisions for the Final Submission

Two decisions shape the final evaluation: which baselines to add, and which one-step number to lead with.

**Baselines to add.** Planned for the final report is a *numerical-solver baseline*. The data were generated with the spectral solver Dedalus [11], and our solver driver is set up as the data's generator was (same equations, boundary conditions and grid; written, not yet run). Restarted from a model's last input frame at the data's own resolution, the solver reproduces the data up to the chaotic growth of tiny differences, so it would show the dataset's own reproducibility limit rather than a fair competitor for surrogates trained for a few hours. The informative comparison is the solver on coarser grids ($256 \times 64$ and $128 \times 32$), which costs less and is less accurate: at what grid and CPU cost does it match each model's accuracy? Comparing against a numerical method at equal accuracy or equal cost follows McGreivy and Hakim [15]. Three cheaper baselines need no GPU training and can be added to the final evaluation: *linear extrapolation* in time ($2\,y_t - y_{t-1}$), which extends persistence by the latest rate of change; *climatology*, the mean training state of the same regime (a vertical profile of each field), the best constant forecast at long lead times; and *dynamic mode decomposition* [16], a linear model fitted by least squares and the classical data-driven baseline in fluid dynamics, which would show how much of the networks' accuracy needs a nonlinear model.

**Which one-step number to lead with.** The Well paper reports one-step error as the mean VRMSE over all one-step windows of the test trajectories (196 per trajectory), in normalised fields; our pipeline check reproduces its published test values this way (Section 2). We call this the all-window mean. VRMSE divides a model's error by how much the true field varies across the grid. During onset the fluid is almost at rest, so the velocity barely varies and the score divides by a number close to zero: errors that are tiny in absolute terms give very large scores, and the score swings with errors that do not matter physically. The 50 onset windows are only a quarter of the total, yet they produce most of the all-window mean: 87% of the released CNextU-Net's 0.224 (onset 0.766, developed 0.038), and 96% under the paper protocol, in which the paper's published values (0.6699 for CNextU-Net, 0.6566 for TFNO) are given, so those values are dominated by the onset windows as well. The all-window mean therefore mostly measures how a model handles the nearly still start rather than how well it forecasts convection, and it can reverse comparisons (Table 8). We report both: the developed-convection error says more about forecasting the flow, and the all-window mean is the number comparable with the paper's published values.

## 6.2 Issues

Nothing blocks the study: every planned 10% run and evaluation finished, and the pipeline reproduces the paper's test values. Table 9 lists what cost time and money or limits what the results can show. The

| Comparison | All windows | Developed convection |
|---|---|---|
| AViT against CNextU-Net (4 h) | 0.134 against 0.136: level | 0.075 against 0.050: CNextU-Net 1.5 times better |
| Our 10% CNextU-Net against the released one | 0.136 against 0.224: ours looks better | 0.050 against 0.038: the released one is better |
| Persistence against TFNO (4 h) | 0.199 against 0.209: persistence looks as good | 0.226 against 0.109: persistence 2 times worse |

**Table 8:** How the all-window mean can reverse comparisons (physical-field one-step test VRMSE, from Table 3).

main open issue is reading the data fast enough on HPC3 (Fig. 9): more than half of the GPU time each AViT run held was spent waiting for data.

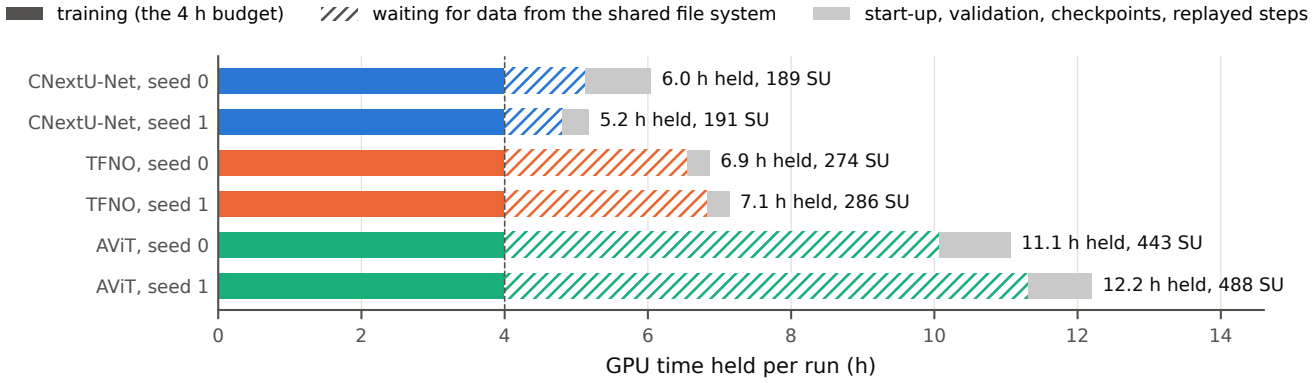



**Figure 9:** Where each 10% run's GPU time went: active training (the 4-hour budget), waiting for data from the shared file system, and other overhead (start-up, validation, checkpoint writes, and steps replayed after a preemption or crash). SU: billed on the paid partition; the CNextU-Net runs started on the free partition, which is not billed.

# 7 Revised Plan

The *core comparison* remains three families, 10% and 100% training data, two seeds, and checkpoints at one and four hours. The 100% runs complete the data axis, so the accuracy–cost comparison of Fig. 6 and Table 7 can be repeated with all the data and the value of ten times more data measured at each budget. The models, data fractions, seeds, budgets and the extensions with their order are as in the abstract.

**Changes since the abstract.**

- **Budget clock.** The 1-hour and 4-hour budgets count active GPU training time, not wall time, because waiting for data on the shared file system would otherwise be measured as training (Section 3). Allocated GPU time, data waits and SU are reported separately so that storage costs remain visible; a 4-hour run held a GPU for 5–12 hours (Fig. 9).
- **Core resource estimate.** The 48 hours of active training are unchanged; the working estimate is approximately 135 allocated GPU-hours for the core study and its supporting work (Appendix Table 16), instead of the abstract's approximately 100, the difference being waits for data. With all 10% work complete, Slurm accounting gives 72.6 allocated GPU-hours: 54.2 on the paid partition, billed 2,160 SU (40 SU per allocated hour for jobs with 8 cores, 36 for the 4-core pipeline check), and 18.3 on the free partition, which is not billed.

| Area | Issue | Effect so far | Status |
|---|---|---|---|
| Data on HPC3 | The shared file system (`/pub`) delivers at most ∼98 training samples per second into one node and stalls for 5–22 minutes at a time, on all nodes at once | TFNO and AViT need 116 and 330 samples/s; per 4-hour run they waited 2.5–2.8 h and 6.1–7.3 h for data (Fig. 9) | **Open.** Assess node-local staging before the 100% runs, whose 294 GB training split no longer fits in a node's page cache |
| | File-system errors (`Errno 70`) | Crashed both AViT runs at the same moment on two nodes, and two learning-rate-search runs | **Mitigated.** Runs resume from their last checkpoint |
| | The upstream loader reopened every file for each sample | The first replication attempt ran at 5.8–47 s per step instead of 0.55–1.42 s | **Resolved.** Our loader keeps files open (same samples bit for bit, 1.2–1.4 times faster) and uses more workers |
| | The files store a uniform placeholder for the vertical coordinate | Boundary checks, spectra and the Nusselt number would be wrong | **Resolved.** Nodes identified from the data (Appendix C) |
| Compute on HPC3 | The free GPU partition preempts jobs, and its A100s were often held for higher-priority jobs | Three 10% runs were preempted; jobs waited without starting | **Worked around.** Training and evaluation moved to the paid partition (class account) |
| | Queue waits on the paid partition | Median about 3 h; up to 12 h before training and 15 h before evaluation | **Open.** The 100% runs need about a day of slack |
| | Allocated time is billed, including data waits | 2,160 SU of the shared class allocation so far; the 100% runs need about 2,400 more | **Open.** Falls if staging removes the waits |
| Training | TFNO's Fourier layers fail in bf16 | Every model trains in fp32, which favours TFNO: bf16 would speed up a training step 1.7 times for CNextU-Net and 2.2 times for AViT | **Accepted** for this comparison; stated as a caveat |
| | The learning rate $10^{-3}$ is the largest tried, from a 20-minute search | The optimum may be higher; CNextU-Net's validation error peaks near one hour, where its two seeds differ most | **Accepted**; stated as a caveat |
| Evaluation | VRMSE is ill-conditioned during onset, where the velocity is nearly zero | Persistence beats every model on the four-field mean during onset | **Handled.** Onset and developed convection reported separately, and per field |
| | Models are trained only on one-step targets | AViT's rollouts are no better than persistence after 30 steps; no model keeps the heat transport right over a rollout | **Open.** Rollout-aware training is outside the study's fixed recipe |
| | The released TFNO was trained with a bug that skipped its Fourier layers; the paper's validation values cannot be reproduced | Only the released CNextU-Net is a valid reference, compared on the test split | **Documented** (Appendix D) |

**Table 9:** Issues met so far, their effect and their status.

- **Baselines.** Persistence is added as the baseline that costs nothing to train. A Dedalus [11] solver baseline on coarser grids is planned after the 100% runs (CPUs on the free partition, so no SU; it first needs a feasibility check). It would set the models' accuracy and forecast time against the solver's for the same 30-step forecasts. Linear extrapolation, climatology and dynamic mode decomposition are candidate baselines that need no GPU training (Section 1).
- **Pipeline check.** It found that the released TFNO was trained with a bug that skipped its Fourier layers and reproduces its published value only with that forward pass. The released CNextU-Net is therefore the only valid released reference; our TFNO uses the fixed code.
- **Training settings fixed.** Peak learning rate $10^{-3}$ for every model, from the equal-budget search (the paper's lower rates suit its 12-hour runs, not our budgets); batch 64; fp32 for every model, because TFNO's Fourier layers fail in bf16.
- **Evaluation.** Report the paper protocol for reference comparisons alongside the physical-field VRMSE, separate onset from developed convection, and report the failure horizon at thresholds 1, 2 and 5 instead of one predefined threshold. Error by Rayleigh number and heat transport (Nusselt number) are added (Appendix F).
- **Optional, outside the estimate.** 24-hour runs of CNextU-Net and TFNO with the paper's learning rates (roughly 60–90 allocated GPU-hours; not an exact replication of the paper's 12-hour H100 runs). A second draw of the 10% subset (one run per model, about 25 allocated GPU-hours) would measure how much the choice of trajectories matters, and batched inference timing can be added to the 100% evaluations at little cost.

**Remaining deliverables.** The 10% evaluation is complete for all three models and both seeds, including failure horizons, spectra, training cost, inference time and memory. Remaining for the final report: the 100% runs and the data–compute comparison (10% against 100% at 1 and 4 hours), with accuracy–cost curves at both data fractions; the Rayleigh-number and heat-transport analyses for the 100% runs; and their flow snapshots.

**Next steps, in priority order.** (1) Improve data-loading robustness and assess node-local staging; (2) complete the six 100% runs (estimated approximately 2,400 SU) and the core data–compute comparison; (3) run the numerical-solver baseline on CPUs; (4) complete the promised metrics and the reproducible final report. Only after core completion, consider the abstract's extensions in their original order: input-noise robustness, a third seed, a 30% fraction, and ConvLSTM. The optional runs above should not delay the core deliverables.

## Appendix A: Physical System and Rayleigh–Prandtl Regimes

The data solve the two-dimensional Boussinesq equations for buoyancy $b$ (the temperature variable), velocity $(u, w)$ and pressure $p$ in a $4 \times 1$ box, periodic in $x$, between no-slip walls held at $b = 1$ (bottom) and $b = 0$ (top) [13]. A regime is set by two dimensionless numbers,

$$\mathrm{Ra} = \frac{g\alpha\,\Delta T\,H^3}{\nu\kappa}, \qquad \mathrm{Pr} = \frac{\nu}{\kappa}, \tag{1}$$

with gravitational acceleration $g$, thermal expansion coefficient $\alpha$, temperature difference $\Delta T$ between the walls, layer height $H$, kinematic viscosity $\nu$ and thermal diffusivity $\kappa$. The Rayleigh number measures the buoyancy forcing against viscous and thermal diffusion: larger values give stronger convection, with finer plumes and thinner boundary layers (Fig. 1). The Prandtl number measures how fast momentum diffuses relative to heat (about 0.7 for air and 7 for water). In the data's units $\kappa = (\mathrm{Ra}\,\mathrm{Pr})^{-1/2}$ and $\nu = (\mathrm{Ra}/\mathrm{Pr})^{-1/2}$.

The dataset crosses five Rayleigh numbers, $\mathrm{Ra} \in \{10^6, 10^7, 10^8, 10^9, 10^{10}\}$, with seven Prandtl numbers, $\mathrm{Pr} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10\}$, giving 35 regimes. Each trajectory starts from rest, with a linear (purely conductive) buoyancy profile plus a small random perturbation. In each regime, five profile amplitudes ($\delta b_0 \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$) and ten noise seeds (40–49) give 50 trajectories, 1,750 in all; trajectories of one regime share its physics and differ only in their initial conditions. Each has 200 frames on a $512 \times 128$ grid.

## Appendix B: Detailed Training Recipe

Table 10 sets our recipe beside the one The Well used for its published models. The reference settings come from the paper (learning rates, 12-hour H100 budget) and from The Well's training configuration; where the two disagree, we follow the configuration. The paragraphs below explain each row.

| | This study | The Well paper / code |
|---|---|---|
| Activation | GELU (all three models) | GELU (same model code) |
| Optimizer | AdamW, $\beta = (0.9, 0.999)$, weight decay $10^{-4}$, no clipping | AdamW, weight decay $10^{-4}$ in the code (0.01 in the text), no clipping |
| Loss | one-step MSE, normalised units, equal field weights | same |
| Batch size, precision | 64, fp32 | 64, fp32 |
| Learning rate | $10^{-3}$ for all, from our search over $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}\}$ | per model from $\{10^{-4}, 5\cdot10^{-4}, 10^{-3}, 5\cdot10^{-3}, 10^{-2}\}$: $5\cdot10^{-4}$ (CNextU-Net), $10^{-4}$ (TFNO) |
| Schedule | 500 warmup steps, cosine to 0 over the 4-hour budget | 5 warmup epochs, cosine to 0.1·lr over 200 epochs, stopped at 12 h |
| Budget, hardware | 1 h and 4 h of active GPU training on one A100 | 12 h on one H100, best validation epoch |
| Training data | 10% of the trajectories (100% next) | 100% |

**Table 10:** Training recipe: ours against the paper's.

**Task and loss.** Each training example is a window of five consecutive frames: the model sees four and predicts the fifth. All four fields are normalised with the mean and standard deviation of The Well's full training split (the same statistics for every subset), and the loss is the mean squared error over all grid points, with equal weight for each field. At 10% data an epoch is 27,440 windows (140 trajectories $\times$ 196); in four hours of active GPU training CNextU-Net passes through it about 28 times, TFNO 61 times and AViT 175 times.

**Optimizer.** AdamW with $\beta = (0.9, 0.999)$ and weight decay $10^{-4}$, without gradient clipping, as in The Well's configuration (the paper's text gives a weight decay of 0.01). Every model uses batch 64 in fp32 (Section 3).

**Learning-rate schedule.** The rate rises linearly from almost zero to its peak over the first 500 optimizer steps (about 10 minutes of training for CNextU-Net, 5 for TFNO and 2 for AViT), then follows a cosine down to zero. The cosine runs over active GPU training time rather than over steps, because the number of steps a model takes in four hours is not known in advance; this way the four-hour checkpoint is exactly the end of the schedule. The one-hour checkpoint is an intermediate state of the same run, with the rate at about 85% of its peak. The Well's schedule counts epochs instead: the rate rises linearly from 0.1·lr to lr over 5 epochs, then follows a cosine down to 0.1·lr at epoch 200.

**Checkpoints and seeds.** We evaluate the checkpoints taken at exactly one and four hours, without choosing among checkpoints by validation score; the paper kept its best validation epoch. During training, 64 fixed, evenly spaced validation windows are scored every 30 minutes of training time, for monitoring only (the training curves, Fig. 3). A seed sets the initial weights and the order in which training windows are drawn; both seeds use the same 10% subset.

**Learning-rate search.** The Well chose each model's rate from $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$ for its 12-hour runs: $5 \cdot 10^{-4}$ for CNextU-Net and $10^{-4}$ for TFNO. For our shorter budgets we ran an equal-budget search: each model at $10^{-4}$, $5 \cdot 10^{-4}$ and $10^{-3}$, on the 10% subset with seed 0, for 20 minutes of active GPU training each (nine runs). The runs used the study's recipe with two changes forced by the short budget: 100 warmup steps instead of 500 (500 would be 60% of CNextU-Net's roughly 850 steps), and 512 validation windows instead of 64, so that the choice does not rest on a few windows. Each model takes the rate with the lowest validation VRMSE at 20 minutes (Table 11); the test split was not used. $10^{-3}$ won for all three models. A 20-minute search can favour rates that learn fast early over those that are best after four hours; this bias is the same for every model.

| Validation VRMSE after 20 min | $10^{-4}$ | $5 \cdot 10^{-4}$ | $10^{-3}$ |
|---|---|---|---|
| CNextU-Net | 0.841 | 0.469 | **0.408** |
| TFNO | 0.603 | 0.358 | **0.302** |
| AViT | 0.646 | 0.332 | **0.257** |

**Table 11:** Learning-rate search.

**Released CNextU-Net's schedule.** The paper does not report how many epochs its 12-hour runs completed. On our A100, CNextU-Net trains on about 54 samples per second; with an H100 at most three times faster, 12 hours give at most about 7 million samples, or about 25 passes over the 274,400 windows of the full training split. The released CNextU-Net therefore completed at most about 25 of its 200 scheduled epochs, and its learning rate was still about 98% of its peak when training stopped, whereas ours decays to zero within the budget. The gap between it and our 10%, four-hour CNextU-Net is therefore not an estimate of what ten times more data buys.

**Software.** PyTorch 2.8.0 with CUDA 12.8 and Python 3.13. The model code is The Well's, with TFNO's layers from neuraloperator 0.3.0.

## Appendix C: Metric Details

**Onset and developed convection.** Every trajectory starts from fluid at rest: the velocity is exactly zero at frame 0, and the temperature falls linearly from the hot bottom wall to the cold top wall, plus a little random noise. This state is unstable, so the noise grows. At first the fluid barely moves; then plumes form; after a while the flow settles into steady, ongoing convection in which plumes keep forming,

rising and mixing, much as a pot of water heated from below sits still at first and later churns steadily. We call the early phase *onset* and the later one *developed convection*. In practice, a one-step window counts as onset if its first input frame is before frame 50: the first 12.5 time units, or 50 of the 196 windows of each trajectory. The cut is where the released CNextU-Net's error levels off on the validation split: its physical-field VRMSE is 1.65, 1.36, 0.71, 0.23 and 0.08 for windows starting at frames 0–9, 10–19, 20–29, 30–39 and 40–49, and at most 0.06 from frame 50 on. It is not a physical transition checked for every trajectory: convection starts later at low Rayleigh numbers and sooner at high ones.

**Why VRMSE is large during onset.** VRMSE divides a model's error by how much the true field varies across the grid. During onset the velocity is almost zero everywhere, so it barely varies, and the score divides by almost zero: errors far too small to matter give large scores. Buoyancy and pressure vary strongly across the layer from the start, so they are not affected. What keeps these velocity scores from growing without limit is the tiny constant $\varepsilon = 10^{-7}$, which the formula adds to the denominator so that it never divides by zero.

**How the two variants differ.** The paper protocol rescales each field before scoring: it subtracts the field's training mean and divides by its training standard deviation $\sigma_f$ (0.249, 0.149, 0.208 and 0.180 for $b$, $p$, $u$ and $w$), which gives the units the models are trained in. This makes the error and the frame's variation larger by the same factor, which cancels out, so on its own the rescaling changes nothing. The exception is $\varepsilon$: it is a fixed number in the formula, so it is not rescaled, and compared with the rescaled fields it is 16 to 45 times smaller (the factor is $1/\sigma_f^2$). This matters only when the true frame barely varies, as the velocity does during onset, when its variation in physical units is about as small as $\varepsilon$ itself. In physical fields $\varepsilon$ is then a noticeable part of the denominator and holds the score down; in the rescaled fields it is too small to do so, and the same small error gives a much larger score. Once convection has developed, the flow varies far more than $\varepsilon$ in both units, and the two give the same score. Table 12 shows this for the released CNextU-Net: the same predictions score 0.224 or 0.669 depending only on the units.

| One-step windows | Physical-field VRMSE | Paper protocol |
|---|---|---|
| Onset (first input frame $< 50$; 50 of 196 per trajectory) | 0.766 | 2.510 |
| Developed convection (first input frame $\geq 50$) | 0.0382 | 0.0383 |
| All windows | 0.224 | 0.669 (published 0.6699) |
| Share of the all-window mean that comes from onset | 87% | 96% |

**Table 12:** The released CNextU-Net's one-step test VRMSE scored both ways. The predictions are the same; only the units of the fields differ.

**Rollouts.** The paper's rollouts start from frame 0 of each trajectory, so they lie entirely in onset, and it reports the mean normalised VRMSE over steps 6–12 and 13–30. Every model in its rollout table scores "$> 10$" (the released CNextU-Net scores 36 and 35 in our pipeline). Our rollouts start from frames 20, 80 and 140, covering both onset and developed flow, and we report the error at each step and the failure horizon. The paper's frame-0 rollouts are reported as well (Table 4).

**Why we lead with the physical-field VRMSE.** First, it was chosen before any training: our evaluation plan specified VRMSE on the stored fields from the start, and the evaluation settings were frozen when the training runs began. How the paper computed its values was only established later, during the pipeline check, and changing the headline metric after seeing results would be a choice made after the fact. Second, it scores the fields as the simulation produced them; normalisation is a device for training. Third, because $\varepsilon$ holds its onset scores down, the onset windows dominate the all-window mean less (87% against 96%), although they still dominate it, which is why we also report onset and developed convection separately. Neither variant scores onset well, since $\varepsilon$ is only there to avoid dividing by zero, not to decide how small an error is too small to matter. Once convection has developed the two agree, and the paper protocol is reported wherever we compare with the paper (Tables 2, 3 and 4).

**Vertical grid.** The files store evenly spaced $y$ coordinates, but the values lie on the 128 Chebyshev–Gauss nodes of the solver's grid. With these nodes the data satisfy the boundary conditions: the Chebyshev interpolant gives $b = 1$ and 0 at the walls to within $6 \times 10^{-10}$, the velocity vanishes there to within $9 \times 10^{-8}$ of its root-mean-square value, and the conductive part of the Nusselt number (the volume-averaged $-\partial b/\partial y$) is 1 to within $3 \times 10^{-9}$; evenly spaced and Gauss–Lobatto nodes fail these checks. The spectrum and the Nusselt number use these nodes; the models and VRMSE treat the 128 rows as an image and use no coordinates.

**Kinetic-energy spectrum.** At each grid row, the two velocity components are Fourier-transformed along the periodic $x$ axis; $E(k_x)$ is the one-sided energy $\frac{1}{2}(|\hat{u}|^2 + |\hat{w}|^2)$ at $k_x = 2\pi n/L_x$ ($L_x = 4$), averaged over the Chebyshev-spaced rows with quadrature weights, so that summing over $k_x$ gives the domain-mean kinetic energy. Spectra are averaged over the rollouts of each regime (5 trajectories $\times$ 3 start frames) at steps 1, 10 and 30, separately for predictions and truth.

**Inference time.** Measured as a scientist would use a model: one forecast at a time (batch 1), fp32, on one A100 80 GB, with the input already on the GPU. After 10 warm-up predictions we time 50 one-step predictions (reported per step) and one full 30-step autoregressive rollout, with a CUDA synchronisation before each clock stops. At batch 1 the GPU is far from fully used, so this latency includes per-call overhead as well as computation. For many forecasts at once, such as ensembles, we also report the forward-pass time per sample in a batch of 64, measured during profiling (fp32, with gradient tracking as in training, so an upper bound for inference).

## Appendix D: TFNO Forward Pass and the Released TFNO

In The Well's code, commit `2d8d7eaf` (2024-06-03, "Added gradient checkpointing") called TFNO's Fourier blocks without keeping their output, so the model computed only its lifting and projection layers; commit `e4301602` (2025-11-17, "actually use the TFNO blocks") restored the assignment (`x =` before the block call) [12]. The paper's TFNO results and the released TFNO checkpoint (uploaded 2025-03-28) date from that period. For the pipeline check we scored the released TFNO weights with the forward pass they were trained with (Fourier blocks skipped): it reproduces the published value, while with the corrected forward pass the same weights score ≈10.5. Our own runs use The Well at commit `e63dec8`, which includes the fix. Upstream issue #49 reports related inconsistencies between released checkpoints and published numbers [14].

| Model | Paper-protocol one-step VRMSE (published) | Physical-field one-step all / onset / developed | Rollout from frame 0, steps 6–12 / 13–30 |
|---|---|---|---|
| Released TFNO (paper-era forward) | 0.6634 (0.6566) | 0.300 / 0.623 / 0.190 | 142 / 169 (published > 10) |
| Our TFNO (10% data, 4 h) | 0.488 | 0.209 / 0.502 / 0.109 | 51 / 77 |
| Persistence | 0.202 | 0.199 / 0.121 / 0.226 | 0.27 / 0.50 |

**Table 13:** The released TFNO, scored by our pipeline on the test split, against our TFNO (seed mean) and persistence.

**Why the released TFNO tracks persistence** (Fig. 2). With its Fourier blocks skipped, the model is only its lifting and projection layers, which act on each grid point separately: every output pixel is computed from the same pixel's four input frames (and at most its position), with no information from neighbouring pixels. Such a model cannot move structures, such as rising plumes or flow carried by the convection rolls, so what it learns is close to copying each pixel's last value with a small correction from its recent trend: persistence with a little extrapolation. In developed convection it is therefore only slightly better than persistence (physical-field VRMSE 0.190 against 0.226). During onset the fields barely change, so copying is nearly exact, while the model's small per-pixel adjustments are divided by

a near-zero spatial variance and become large relative errors. The published all-window value (0.6566) hides this, because every model scores poorly on the onset windows. Our TFNO, which uses its Fourier layers, is better than the released TFNO everywhere (developed 0.109 against 0.190; paper protocol 0.488 against 0.663) despite 10% of the data and 4 hours.

## Appendix E: Resource Accounting

**Timer.** Active training time is measured per optimizer step with a host-side monotonic clock (Python's `perf_counter`), from the moment the data loader returns the batch to just after a CUDA synchronisation (`torch.cuda.synchronize`) following the optimizer update. It includes the host-to-device copy of the batch (pinned memory, non-blocking), setting the learning rate, the forward pass, the loss, the backward pass, the optimizer step and the host-side launch overhead of these operations. It excludes the wait for the next batch (logged as data-wait time), reading the loss value, logging, validation, checkpoint writes and start-up. After a crash the clock resumes from the last saved checkpoint, so lost steps are not counted. A milestone checkpoint is written after the first step at which the cumulative time reaches 1 h or 4 h (overshoot below one step).

**Validation sample.** The 64 validation windows are fixed and evenly spaced over the 34,300 one-step windows of the validation split (175 trajectories × 196), taken in the dataset's order (regime, trajectory, time): indices round(linspace(0, 34299, 64)). They cover all 35 regimes (one or two windows each) and 64 different trajectories; 18 of them (28%) are onset windows, close to the 26% of all windows. The learning-rate search used 512 windows chosen the same way (all 35 regimes, all 175 trajectories, 25% onset); the two samples share 8 windows. This monitoring sample does not replace the full test evaluation.

**Shared storage.** Besides its limited rate (∼98 training samples per second into one node, Fig. 10), the cluster's shared file system stalls for 5–22 minutes at a time, on all nodes at once. Both add to the time a run waits for data (Table 15).

**Billing.** The cluster charges for allocated time on its paid partition, in *service units* (SU): 32 SU per GPU-hour plus 1 SU per core-hour (40 SU per hour for one A100 and 8 cores); the free partition is not billed. Runs that started on the free partition and resumed on the paid one are billed only for the paid part. Time spent waiting in the queue before a job starts is neither allocated nor billed.

**Pilot against actual.** The pilots predicted 4 h of GPU training to take 6.0 / 8.0 / 9.7 h of wall time (CNextU-Net / TFNO / AViT) at the stall rate observed then; the 10% runs took 5.2–6.0 / 6.9–7.1 / 11.1–12.2 h.

| | Validation VRMSE (seed 0 / 1) | | Epochs | Allocated time | SU |
|---|---|---|---|---|---|
| Model | 1 h | 4 h | in 4 h | per run | per run |
| CNextU-Net | 1.043 / 0.772 | 0.167 / 0.178 | 28 | 5.2–6.0 h | 189–191 |
| TFNO | 0.508 / 0.524 | 0.310 / 0.325 | 61 | 6.9–7.1 h | 274–286 |
| AViT | 0.468 / 0.418 | 0.161 / 0.167 | 175 | 11.1–12.2 h | 443–488 |

**Table 14:** The 10% runs (140 training trajectories). Validation: 64 fixed windows, physical-field VRMSE. SU covers the paid partition only: CNextU-Net's first attempts ran on the free partition, so its SU corresponds to 4.8 h of its 5.2–6.0 h.

| Hours per run | CNextU-Net | TFNO | AViT |
|---|---|---|---|
| *10% runs, measured* | | | |
| GPU training (the budget) | 4.0 | 4.0 | 4.0 |
| Waiting for data | 0.8–1.1 | 2.5–2.8 | 6.1–7.3 |
| Start-up, and steps replayed after a preemption or crash[a] | 0.13–0.65 | 0.14–0.15 | 0.68–0.69 |
| Validation (64 windows every 30 minutes) | 0.01–0.02 | 0.02–0.10 | 0.18 |
| Checkpoint writes | 0.23–0.24 | 0.08–0.15 | 0.03–0.13 |
| **Allocated time of training** | **5.2–6.0** | **6.9–7.1** | **11.1–12.2** |
| Test evaluation job (1-hour and 4-hour checkpoints) | 0.87–0.95 | 0.57 | 0.34–0.38 |
| *100% runs, projected* | | | |
| Waiting for data | 1.0–2.0 | 3.0–4.5 | 9.5–12.5 |
| **Allocated time of training** | **5.5–6.5** | **7.5–9** | **14–17** |
| Test evaluation job | ∼0.9 | ∼0.6 | ∼0.4 |

**Table 15:** Time per run (range over the two seeds). [a]A run that starts once spends about 0.15 h in start-up (environment, data index, model build); the larger values belong to runs with two attempts, which also replayed the steps since their last saved checkpoint. Projection for 100%: the shared file system delivers at most ∼98 samples/s, below TFNO's (116) and AViT's (330) demand, and the full training split (∼290 GB) no longer fits in a node's page cache; start-up, validation, checkpoint writes and evaluation as measured. Queue waits before a job starts depend on the cluster's load and are not included (10% runs: 0.1–8.8 h before training, 1–14 h before evaluation).

| Component | Allocated GPU-hours |
|---|---|
| Six 10% runs, completed (24.0 of them active training) | 49.8 |
| Learning-rate search, profiling, pipeline check, environment checks, first replication try | 18.5 |
| Test evaluation of the six 10% runs and persistence (completed) | 4.2 |
| Six 100% runs (projected, 24.0 active; Appendix E) | ∼54–65 |
| Test evaluation of the 100% runs | ∼4.5 |
| Core study | ∼131–142 |

**Table 16:** Allocated GPU-hours of the core study: completed parts from Slurm accounting (2026-11-11), the rest projected. No contingency is included; the abstract's ∼100 hours included ∼30, and a similar margin would bring the total to ∼160–170.

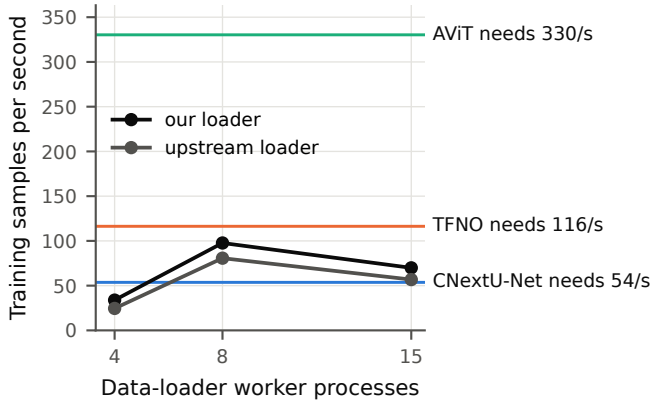



**Figure 10:** Training samples per second read from the shared file system by one node, against the rate at which each model consumes them on the GPU (horizontal lines).

## Appendix F: Additional Analysis

Two analyses go beyond the abstract's metrics: how the error depends on the Rayleigh number, and whether the models reproduce the heat that convection carries across the layer.

**Error by Rayleigh number** (Fig. 11a, b). Each Rayleigh number groups seven Prandtl numbers (35 test trajectories). The one-step error in developed convection grows four- to tenfold from $\mathrm{Ra} = 10^6$ to $10^{10}$ (CNextU-Net 0.016 to 0.111, TFNO 0.042 to 0.191, AViT 0.016 to 0.154), as convection becomes more turbulent, with finer plumes and thinner boundary layers; persistence's error grows too (0.098 to 0.351), because the flow changes more between frames. The ranking is the same at every Rayleigh number, except that AViT ties CNextU-Net at $10^6$. The released CNextU-Net, trained on all the data, matches ours at $10^6$ and $10^7$ (0.018 and 0.017, against 0.016 and 0.020) and is better only at higher Rayleigh numbers (0.085 against 0.111 at $10^{10}$): the 10% model falls short where the flow is most complex, with the caveat that the two runs also differ in schedule and hardware. After 30 steps from developed flow, CNextU-Net and TFNO stay below persistence at every Rayleigh number, while AViT is worse than persistence up to $10^8$ (0.81 against 0.66 at $10^6$) and better only at $10^9$ and $10^{10}$. Over all three start frames, the fraction of CNextU-Net's rollouts that stay below VRMSE 1 instead rises with Rayleigh number (0.67 at $10^6$, 0.98 at $10^{10}$). This is consistent with convection starting later at low Rayleigh number, so that more rollouts from frame 20 begin in the near-quiescent onset, where relative errors are large; the same effect underlies the released model's Rayleigh-number dependence (Section 2).

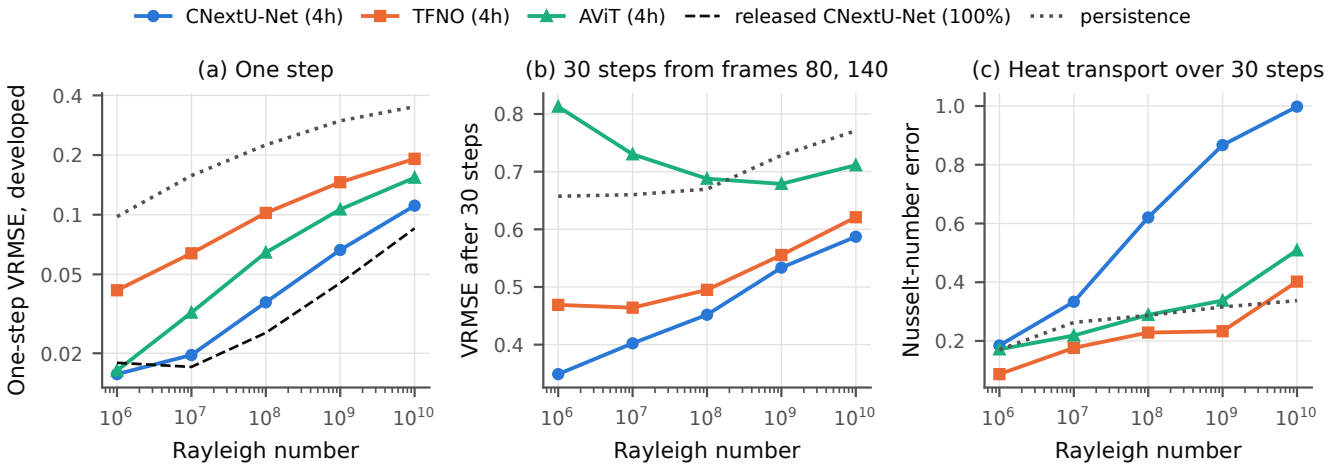



**Figure 11:** Error by Rayleigh number, four-hour models (seed mean; each point averages seven Prandtl numbers, 35 test trajectories). (a) One-step VRMSE in developed convection. (b) VRMSE after 30 steps from frames 80 and 140. (c) Median relative error of the Nusselt number averaged over those 30-step rollouts. Dashed: released CNextU-Net (100% data); dotted: persistence.

**Heat transport** (Table 17, Fig. 11c). The Nusselt number is the heat carried across the layer relative to conduction alone. Volume-averaged, and in the units of the data (diffusivity $\kappa = (\mathrm{Ra}\,\mathrm{Pr})^{-1/2}$, unit height and temperature difference), it is $\mathrm{Nu} = 1 + \sqrt{\mathrm{Ra}\,\mathrm{Pr}}\,\langle w\,b\rangle$, computed for every predicted and true frame of each rollout. Two properties shape the comparison. The factor $\sqrt{\mathrm{Ra}\,\mathrm{Pr}}$, $3 \times 10^4$ to $3 \times 10^5$ at $\mathrm{Ra} = 10^{10}$, amplifies small errors in the correlation between vertical velocity and buoyancy. And the instantaneous value fluctuates strongly: across test trajectories the true Nusselt number one step after frame 80 ranges from $-249$ to 1,363, so relative errors near zero crossings are unbounded and means are dominated by outliers. We therefore report medians, and the error of the Nusselt number averaged over each 30-step rollout from frames 80 and 140; its sign shows whether a model predicts too much or too little heat transport.

After one step the four-hour CNextU-Net reproduces the instantaneous Nusselt number to within 2%

| | One step | Averaged over 30 steps | |
|---|---|---|---|
| Model (10% data, checkpoint) | median \|error\| | median \|error\| | median signed error |
| CNextU-Net, 4 h | **0.020** | 0.485 | −0.015 |
| TFNO, 4 h | 0.072 | **0.196** | +0.064 |
| AViT, 4 h | 0.037 | 0.275 | −0.167 |
| CNextU-Net, 1 h | 0.265 | 5.479 | +5.451 |
| TFNO, 1 h | 0.285 | 0.906 | +0.205 |
| AViT, 1 h | **0.081** | **0.353** | +0.138 |
| Persistence | 0.076 | 0.267 | +0.023 |

**Table 17:** Relative error of the volume-averaged Nusselt number on the 350 rollouts from frames 80 and 140 (175 trajectories × 2), median over rollouts, mean over the two seeds. One step: the instantaneous value after the first step. Signed: negative means too little heat transport. Bold: best of our models at each budget.

(median), better than AViT (3.7%), TFNO (7.2%) and persistence (7.6%), and it is the best or tied at every Rayleigh number. Over a 30-step rollout the picture changes. TFNO comes closest to the true heat transport (0.20, against 0.27 for persistence). AViT systematically predicts too little (median −17%, down to −43% at $\mathrm{Ra} = 10^{10}$). CNextU-Net is unbiased on average (−1.5%) but has the largest spread (0.49), which grows with Rayleigh number to 1.0 at $10^{10}$. Its excess small-scale energy (Fig. 7) could create spurious correlations between $w$ and $b$ that the factor $\sqrt{\mathrm{Ra}\,\mathrm{Pr}}$ amplifies; this is consistent with the data but not established. After one hour CNextU-Net predicts about six times the true heat transport. Heat transport is therefore largely a negative result: a low one-step VRMSE does not guarantee the right heat flux over a rollout, and the MSE objective does not constrain it.

## Appendix G: Additional Figures

Flow snapshots at $\mathrm{Ra} = 10^{10}$ (Fig. 12).

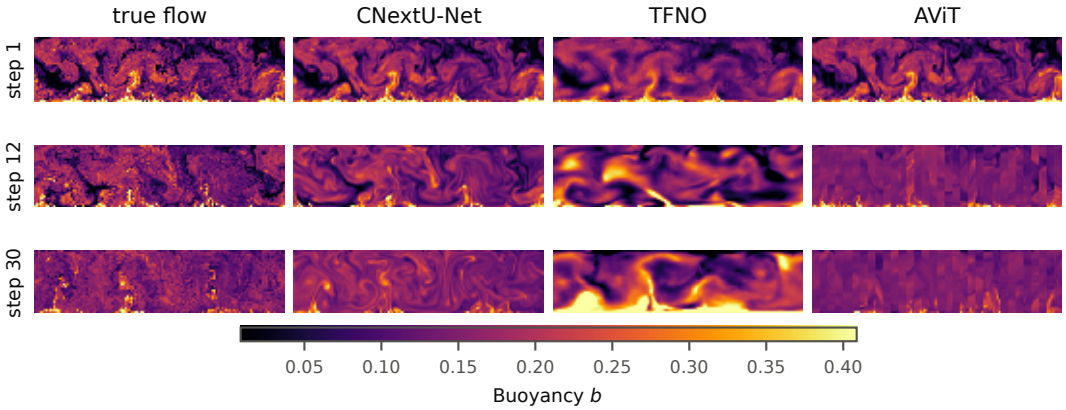



**Figure 12:** As Fig. 8, at $\mathrm{Ra} = 10^{10}$, $\mathrm{Pr} = 1$.

## References


[1] Ohana, R., et al. "The Well: a Large-Scale Collection of Diverse Physics Simulations for Machine Learning." NeurIPS, Datasets and Benchmarks Track, 2024.
https://arxiv.org/abs/2412.00568

[2] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. "A ConvNet for the 2020s." CVPR, 2022.
https://arxiv.org/abs/2201.03545

[3] Kossaifi, J., Kovachki, N., Azizzadenesheli, K., and Anandkumar, A. "Multi-Grid Tensorized Fourier Neural Operator for High-Resolution PDEs." TMLR, 2024.
https://arxiv.org/abs/2310.00120

[4] McCabe, M., et al. "Multiple Physics Pretraining for Spatiotemporal Surrogate Models." NeurIPS, 2024.
https://arxiv.org/abs/2310.02994

[5] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. "Fourier Neural Operator for Parametric Partial Differential Equations." ICLR, 2021.
https://arxiv.org/abs/2010.08895

[6] Gupta, J. K., and Brandstetter, J. "Towards Multi-spatiotemporal-scale Generalized PDE Modeling." arXiv:2209.15616, 2022.
https://arxiv.org/abs/2209.15616

[7] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F. A., Bengio, Y., and Courville, A. "On the Spectral Bias of Neural Networks." ICML, 2019.
https://arxiv.org/abs/1806.08734

[8] Lippe, P., Veeling, B. S., Perdikaris, P., Turner, R. E., and Brandstetter, J. "PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers." NeurIPS, 2023.
https://arxiv.org/abs/2308.05732

[9] Brandstetter, J., Worrall, D., and Welling, M. "Message Passing Neural PDE Solvers." ICLR, 2022.
https://arxiv.org/abs/2202.03376

[10] McCabe, M., Harrington, P., Subramanian, S., and Brown, J. "Towards Stability of Autoregressive Neural Operators." TMLR, 2023.
https://arxiv.org/abs/2306.10619

[11] Burns, K. J., Vasil, G. M., Oishi, J. S., Lecoanet, D., and Brown, B. P. "Dedalus: A flexible framework for numerical simulations with spectral methods." Physical Review Research 2, 023068, 2020.
https://arxiv.org/abs/1905.10388

[12] The Well source code, commits `2d8d7eaf9eb46b96f215abf39c94b6be4bf9ea5c` (2024-06-03) and `e4301602e57f2dddeaa87856dda49bbba47c663b` (2025-11-17).
https://github.com/PolymathicAI/the_well/commit/2d8d7eaf9eb46b96f215abf39c94b6be4bf9ea5c

https://github.com/PolymathicAI/the_well/commit/e4301602e57f2dddeaa87856dda49bbba47c663b

[13] The Well, `rayleigh_benard` dataset card, commit `e63dec8`.
https://github.com/PolymathicAI/the_well/blob/e63dec8/datasets/rayleigh_benard/README.md

[14] The Well, issue #49, "Inconsistent Pre-trained Baseline Performance" (open since 2025-04-21).
https://github.com/PolymathicAI/the_well/issues/49

[15] McGreivy, N., and Hakim, A. "Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations." Nature Machine Intelligence 6, 2024.
https://arxiv.org/abs/2407.07218

[16] Schmid, P. J. "Dynamic mode decomposition of numerical and experimental data." Journal of Fluid Mechanics 656, 5–28, 2010.
https://doi.org/10.1017/S0022112010001217